Ejercicios sobre las implementaciones y quicksort

Pagar al partir para no pagar al combinar

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Resolver problemas sobre la lista, la pila y la cola enlazadas, y dar el costo en tiempo y en espacio de cada solución (OA2, OA3).
- Explicar cómo `particionar` deja el pivote en su lugar definitivo, siguiendo las zonas del arreglo (OA1).
- Construir quicksort sobre `particionar` y justificar su costo en el caso balanceado y en el peor caso, en tiempo y en espacio (OA1, OA2).
- Comparar insertion sort, merge sort y quicksort con comparaciones contadas sobre la misma entrada, y escoger uno según los datos (OA1, OA5).

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

La lista, la pila y la cola sobre nodos I

TAD	Operación	Tiempo	Por qué
Lista	`insertar(0, e)`	$\Theta(1)$	se escribe la cabeza
	`agregar(e)`	$\Theta(1)$	se cuelga de `ultimo`
	`obtener(p)` y similares	$\Theta(p)$	se camina hasta p
Pila	`apilar`, `desapilar`, `tope`	$\Theta(1)$	la cima es la cabeza
Cola	`encolar`	$\Theta(1)$	se cuelga de `ultimo`
	`desencolar`, `frente`	$\Theta(1)$	se sale por la cabeza
Las tres	`tamano`, `vacia`	$\Theta(1)$	se lee `n`
	espacio	$\Theta(n)$	dato y puntero por nodo

Ninguna tiene techo: cada elemento reserva su nodo al entrar y lo libera al salir. A cambio, llegar a una posición del medio exige caminar.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Ejercicios en parejas I

(a) La cola doble

Una cola que deja entrar y salir por los dos extremos: `insertarFrente`, `insertarFinal`, `eliminarFrente` y `eliminarFinal`. ¿Qué lista hace falta debajo para que las cuatro sean $\Theta(1)$? ¿Cuál de las cuatro falla con la lista simple que guarda `ultimo`?

(b) La cola sin contador

Si la cola de nodos no guardara `n`, ¿cómo responde `vacia()`? ¿Y `tamano()`? ¿Cuánto cuesta cada una?

Ejercicios en parejas II

(c) Invertir una cola

Déle la vuelta a una cola usando solo una pila auxiliar y las operaciones de los dos contratos: `encolar`, `desencolar`, `frente`, `vacia` de la cola y `apilar`, `desapilar`, `tope`, `vacia` de la pila. ¿Cuánto cuesta en tiempo? ¿Cuánto espacio pide aparte de la cola?

Las tres se escriben sobre los TAD que ya están construidos: la cola y la pila sobre nodos.

(a) La cola doble: dónde falla la lista simple I

Con la simple y `ultimo`, tres de las cuatro operaciones son $\Theta(1)$. La cuarta no.

Operación	Simple con `ultimo`	Por qué
`insertarFrente`	$\Theta(1)$	se escribe la cabeza
`insertarFinal`	$\Theta(1)$	se cuelga de `ultimo`
`eliminarFrente`	$\Theta(1)$	la cabeza pasa al siguiente
`eliminarFinal`	$\Theta(n)$	hay que llegar al penúltimo

Quitar el último deja al penúltimo como nuevo último, y el penúltimo solo se alcanza caminando desde la cabeza. `ultimo` sirve para poner al final, no para quitar de ahí.

(a) La cola doble: dónde falla la lista simple II

La lista doblemente enlazada con cabeza y `ultimo` resuelve la cuarta: el penúltimo es `ultimo->anterior`.

```
// exige !vacia()
void eliminarFinal() {
  assert(!vacia());
  NodoDoble *muerto = ultimo;
  ultimo = ultimo->anterior;
  if (ultimo == NULL) {
    cabeza = NULL;
  } else {
    ultimo->siguiente = NULL;
  }
  delete muerto;
  n = n - 1;
}
```

(a) La cola doble: dónde falla la lista simple III

```
void insertarFrente(Elemento e) {
  NodoDoble *nuevo = new NodoDoble;
  nuevo->dato = e;
  nuevo->anterior = NULL;
  nuevo->siguiente = cabeza;
  if (cabeza == NULL) {
    ultimo = nuevo;
  } else {
    cabeza->anterior = nuevo;
  }
  cabeza = nuevo;
  n = n + 1;
}
```

Las otras dos son el espejo de estas: se cambia `cabeza` por `ultimo` y `siguiente` por `anterior`.

(a) La cola doble: dónde falla la lista simple IV

Una corrida

`insertarFinal(4)`, `insertarFinal(7)`, `insertarFrente(1)`: `frente 1, final 7, tamano 3`.
`eliminarFinal()`: `frente 1, final 4`.
`eliminarFrente()` y `eliminarFinal()`: la cola queda vacía, y `insertarFrente(9)` deja `frente 9, final 9`.

Operación	Doble con `ultimo`	Por qué
las cuatro de los extremos	$\Theta(1)$	a lo sumo cuatro punteros
`frente`, `final`	$\Theta(1)$	se lee `cabeza` o `ultimo`
espacio	$\Theta(n)$	dato y dos punteros por nodo

(a) La cola doble: dónde falla la lista simple V

El caso que rompe la cola doble

Al sacar el único nodo por cualquiera de los dos lados, el otro extremo también debe quedar en `NULL`. Si no, apunta a un nodo liberado.

(b) La cola sin contador I

Sin `n`, la cadena misma dice si hay elementos: la cola está vacía cuando no hay cabeza. Contarlos exige recorrerla.

```
bool vacia() {
  return cabeza == NULL;
}

int tamano() {
  int cuenta = 0;
  Nodo *actual = cabeza;
  while (actual != NULL) {
    cuenta = cuenta + 1;
    actual = actual->siguiente;
  }
  return cuenta;
}
```

(b) La cola sin contador II

Operación	Sin `n`	Por qué
`vacia()`	$\Theta(1)$	compara un puntero
`tamano()`	$\Theta(n)$	visita cada nodo una vez
`encolar`, `desencolar`, `frente`	$\Theta(1)$	no usaban `n`
espacio de la clase	dos punteros	uno menos que antes

Lo que cuesta ahorrar un entero

Un ciclo que pregunte `tamano()` en cada vuelta, como el que imprime la cola dándole la vuelta, pasa de $\Theta(n)$ a $\Theta(n^2)$. Con el contador, la cola gasta un entero y responde en $\Theta(1)$.

(c) Invertir una cola con una pila I

La pila devuelve al revés lo que recibe. Si se le pasa la cola en orden y después se vuelca en la cola, sale invertida.

```
void invertir(Cola &c) {
  Pila p;
  while (!c.vacia()) {
    p.apilar(c.frente());
    c.desencolar();
  }
  while (!p.vacia()) {
    c.encolar(p.tope());
    p.desapilar();
  }
}
```

(c) Invertir una cola con una pila II

Con $\langle 3, 1, 4, 1, 5 \rangle$

Momento	Cola, del frente al final	Pila, del tope al fondo
al empezar	$\langle 3, 1, 4, 1, 5 \rangle$	vacía
tras el primer ciclo	vacía	$\langle 5, 1, 4, 1, 3 \rangle$
tras el segundo ciclo	$\langle 5, 1, 4, 1, 3 \rangle$	vacía

El programa imprime `antes: 3 1 4 1 5` y `despues: 5 1 4 1 3`.

	Costo	Por qué
tiempo	$\Theta(n)$	cada elemento sale y entra una vez en cada TAD, a $\Theta(1)$
espacio aparte	$\Theta(n)$	la pila guarda los n elementos a la vez

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Merge sort paga al combinar I

Merge sort parte la lista por la mitad sin mirar los datos, y todo el trabajo queda para el final: mezclar dos mitades ordenadas recorre las dos.

Pregunta

¿Y si se paga al partir? Si al dividir se deja a la izquierda todo lo pequeño y a la derecha todo lo grande, ¿qué queda por hacer después de ordenar cada lado?

Merge sort paga al combinar II

Respuesta

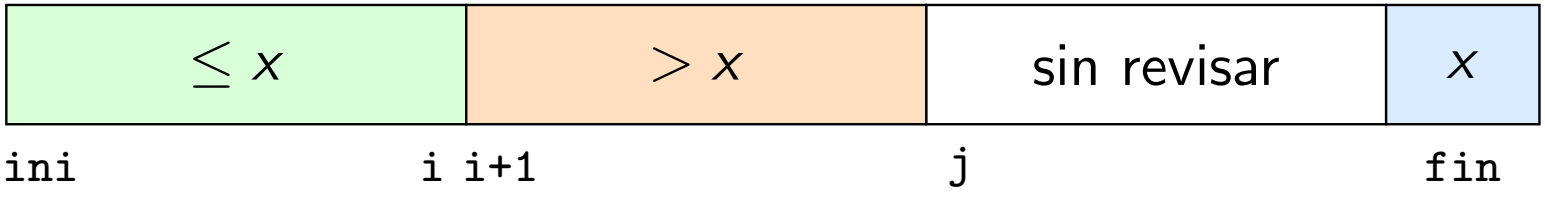
Nada. Si cada elemento de la izquierda es menor o igual que cada elemento de la derecha, ordenar cada lado deja todo el arreglo ordenado, y combinar no cuesta.

Quicksort sigue los tres pasos de dividir, conquistar y combinar sobre un arreglo $a[ini..fin]$ (CLRS, §7.1):

- **Dividir**: escoger un pivote x y reordenar de modo que queden primero los menores o iguales que x, después x y al final los mayores. El pivote queda en una posición q.
- **Conquistar**: ordenar $a[ini..q-1]$ y $a[q+1..fin]$ con el mismo procedimiento.
- **Combinar**: no hay nada que hacer.

Partir alrededor de un valor, sin auxiliar I

Partir una cola alrededor de un valor ya se hizo con una cola auxiliar. Sobre el arreglo se puede hacer en el sitio: el ciclo mantiene cuatro zonas y las hace crecer moviendo dos índices.



- El pivote es el último: $x = a[fin]$.
- `j` recorre de izquierda a derecha y revisa un elemento por vuelta.
- `i` marca el final de la zona de los menores o iguales.
- Al terminar, el pivote se cambia con el primero de los mayores y queda entre las dos zonas.

particionar (CLRS, §7.1) I

```
void intercambiar(Elemento a[], int p, int q) {
  Elemento temp = a[p];
  a[p] = a[q];
  a[q] = temp;
}
```

particionar (CLRS, §7.1) II

```
// Parte a[ini..fin] alrededor de x = a[fin] y
// devuelve la posicion final del pivote. Exige ini <= fin.
int particionar(Elemento a[], int ini, int fin) {
  Elemento x = a[fin];
  int i = ini - 1;
  int j = ini;
  while (j < fin) {
    if (a[j] <= x) {
      i = i + 1;
      intercambiar(a, i, j);
    }
    j = j + 1;
  }
  intercambiar(a, i + 1, fin);
  return i + 1;
}
```

`particionar` (CLRS, §7.1) III

Un elemento mayor que x no se mueve: la zona de los mayores crece sola cuando avanza `j`. Un elemento menor o igual se cambia con el primero de los mayores, y las dos zonas avanzan una casilla.

La traza con $\langle 7, 2, 9, 4, 1, 5 \rangle$ I

$ini = 0$, $fin = 5$, $x = 5$. Cada fila es el estado al evaluar la condición del `while`.

`i`	`j`	$a[0..5]$	$a[j] \leq 5$	Qué pasa
-1	0	7 2 9 4 1 5	no	solo avanza `j`
-1	1	7 2 9 4 1 5	sí	`i` pasa a 0; cambia $a[0]$ y $a[1]$
0	2	2 7 9 4 1 5	no	solo avanza `j`
0	3	2 7 9 4 1 5	sí	`i` pasa a 1; cambia $a[1]$ y $a[3]$
1	4	2 4 9 7 1 5	sí	`i` pasa a 2; cambia $a[2]$ y $a[4]$
2	5	2 4 1 7 9 5		la condición falla: $j = fin$

Al salir se cambia $a[3]$ con $a[5]$: queda $\langle 2, 4, 1, \mathbf{5}, 9, 7 \rangle$ y la función devuelve $q = 3$. Los dos lados no están ordenados, pero el 5 ya está en la casilla que ocupará al final.

Lo que mantiene el ciclo I

Cada vez que se revisa la condición del `while`, el arreglo está repartido en las cuatro zonas del dibujo:

- de `ini` a `i`, los menores o iguales que x;
- de `i+1` a `j-1`, los mayores que x;
- de `j` a `fin-1`, los que faltan por revisar;
- en `fin`, el pivote, que no se mueve durante el ciclo.

Una vuelta revisa $a[j]$

- Si es mayor que x, ya está al final de la zona de los mayores: basta avanzar `j`.
- Si es menor o igual, `i` avanza y el cambio lo lleva al final de los menores; el primero de los mayores pasa al final de su zona.

Lo que mantiene el ciclo II

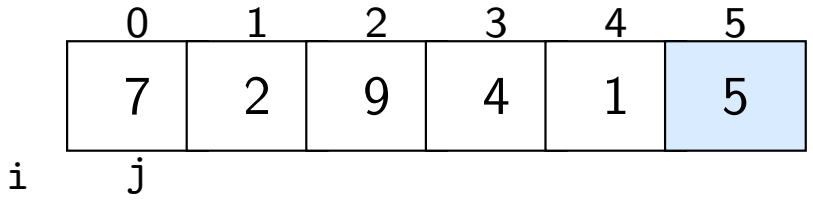
En la traza con $\langle 7, 2, 9, 4, 1, 5 \rangle$, en la fila $i = 1$, $j = 4$ el arreglo es 2 4 | 9 7 | 1 | 5: los menores o iguales, los mayores, lo que falta y el pivote.

Al salir

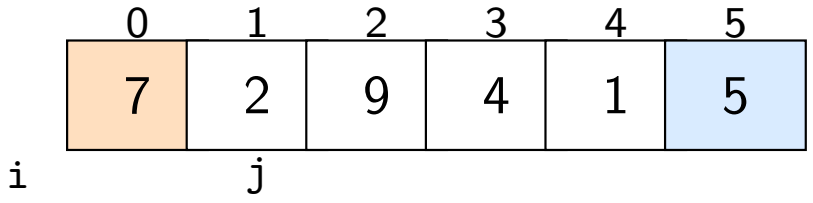
Cuando `j` llega a `fin` ya no queda nada sin revisar. El cambio de $a[i+1]$ con $a[\mathit{fin}]$ pone el pivote entre las dos zonas: a su izquierda todo es menor o igual, a su derecha todo es mayor, y esa casilla es la que ocupará en el arreglo ordenado.

Paso a paso sobre el arreglo I

Verde: menores o iguales que $x = 5$. Naranja: mayores. Blanco: sin revisar. Azul: el pivote.

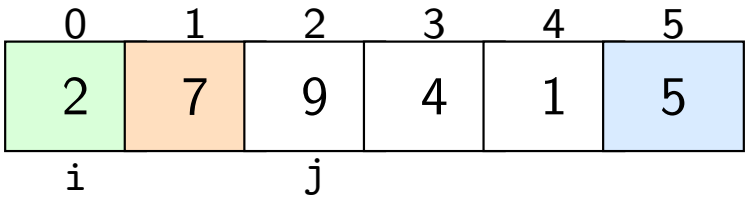


Al empezar: todo está sin revisar. $a[0] = 7 > 5$: solo avanza `j`.

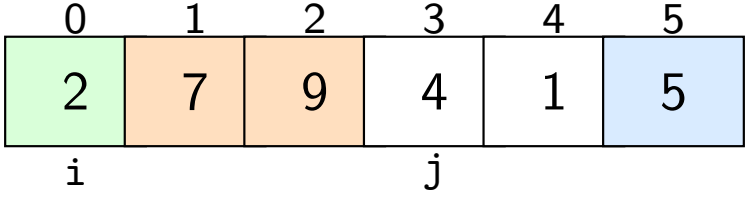


El 7 forma la zona de mayores. $a[1] = 2 \leq 5$: `i` pasa a 0 y se cambian $a[0]$ y $a[1]$.

Paso a paso sobre el arreglo II

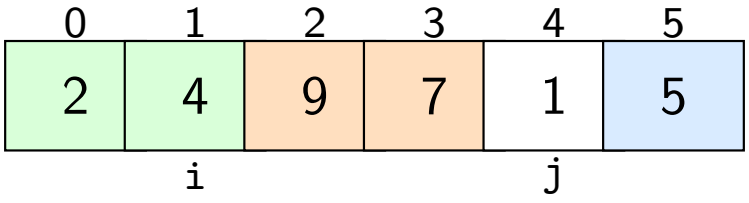


El 2 entró a los menores y el 7 sigue en los mayores. $a[2] = 9 > 5$: solo avanza `j`.

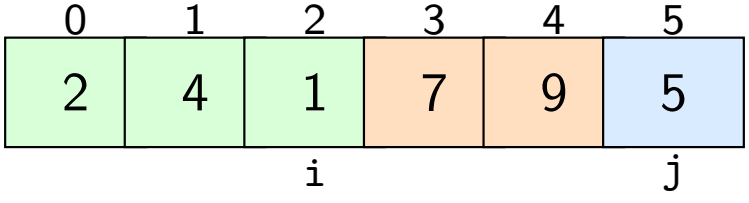


Los mayores son 7 y 9. $a[3] = 4 \leq 5$: `i` pasa a 1 y se cambian $a[1]$ y $a[3]$.

Paso a paso sobre el arreglo III

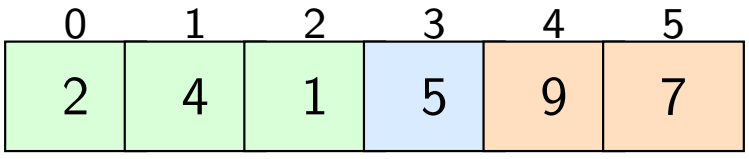


El 4 se unió al 2; el 7 pasó al final de los mayores. $a[4] = 1 \leq 5$: `i` pasa a 2 y se cambian $a[2]$ y $a[4]$.



`j` llegó a `fin`: no queda nada sin revisar. Se cambian $a[3]$ y $a[5]$.

Paso a paso sobre el arreglo IV



El pivote quedó en la casilla 3, entre las dos zonas, y `particionar` devuelve $q = 3$.

quicksort I

```
void quicksort(Elemento a[], int ini, int fin) {
  if (ini < fin) {
    int q = particionar(a, ini, fin);
    quicksort(a, ini, q - 1);
    quicksort(a, q + 1, fin);
  }
}
```

La llamada inicial es `quicksort(a, 0, n - 1)`. Un rango de cero o un elemento no entra al `if`: ya está ordenado.

Sobre $\langle 6, 2, 9, 4, 1, 7, 3, 8 \rangle$

```
quicksort:   1 2 3 4 6 7 8 9
```

quicksort II

Por qué queda ordenado

Cada llamada a `particionar` deja su pivote en la casilla que ocupará al final. Las dos llamadas siguientes trabajan cada una de su lado y no lo cruzan: nada de la izquierda tiene que pasar a la derecha. Cuando los rangos llegan a cero o un elemento, todos los valores ya fueron pivote o quedaron solos en su casilla.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

El costo de `particionar` I

El ciclo da exactamente $fin - ini$ vueltas, con una comparación $a[j] \leq x$ y a lo sumo un intercambio en cada una.

	Costo con $n = fin - ini + 1$	Por qué
comparaciones	$n - 1$	una por vuelta
tiempo	$\Theta(n)$	cada vuelta es $\Theta(1)$
espacio aparte	$\Theta(1)$	`x`, `i`, `j` y `temp`

El costo de quicksort entonces lo decide un solo dato: dónde queda q, porque eso fija el tamaño de las dos llamadas.

El caso balanceado I

Si el pivote cae siempre cerca de la mitad, cada llamada se parte en dos de tamaño $n/2$:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1)$$

Es la recurrencia de merge sort: el árbol de llamadas tiene $\log_2 n$ niveles y cada nivel hace $\Theta(n)$ de trabajo en total.

$$T(n) = \Theta(n \log n)$$

No hace falta la mitad exacta

Si cada partición dejara un décimo de un lado y nueve décimos del otro, el árbol tendría $\log_{10/9} n$ niveles, que sigue siendo $\Theta(\log n)$, y cada nivel costaría a lo sumo $\Theta(n)$: el total sigue en $\Theta(n \log n)$ (CLRS, §7.2).

Pregunta I

¿Qué pasa si el arreglo ya viene ordenado?

Siga `particionar` sobre $\langle 1, 2, 3, 4, 5 \rangle$. ¿Dónde queda el pivote? ¿De qué tamaño son las dos llamadas que siguen?

El peor caso: ya ordenado I

El pivote es el 5, el mayor. Todos los $a[j]$ cumplen $a[j] \leq 5$, así que `i` avanza con `j` y cada intercambio es de una casilla consigo misma. Queda $q = 4$: la izquierda tiene cuatro elementos, todavía ordenados, y la derecha ninguno.

Llamada	Elementos	q	Comparaciones
$a[0.,4]$	5	4	4
$a[0.,3]$	4	3	3
$a[0.,2]$	3	2	2
$a[0.,1]$	2	1	1
			10

Cada llamada quita un solo elemento: el pivote.

El peor caso: ya ordenado II

Teorema

Sobre un arreglo de $n \geq 1$ elementos distintos en orden creciente, quicksort hace $C(n) = n(n-1)/2$ comparaciones, y $C(n) \in \Theta(n^2)$.

Demostración. Se procede de forma directa: primero se plantea la recurrencia de C y se resuelve, y después se acota con testigos.
Desarrollo. En un rango ordenado de $n \geq 2$ elementos el pivote es el máximo, `particionar` hace $n-1$ comparaciones, devuelve $q = \mathit{fin}$ y deja la izquierda igual, ordenada, con $n-1$ elementos. La derecha está vacía. Entonces

$$C(n) = (n-1) + C(n-1), \qquad C(1) = 0,$$

y desplegando, $C(n) = (n-1) + (n-2) + \cdots + 1 + 0 = \dfrac{n(n-1)}{2}$.

El peor caso: ya ordenado III

Cota superior: para $n \geq 1$,

$$\frac{n(n-1)}{2} \leq \frac{n^2}{2},$$

así que $C(n) \leq c\, n^2$ con $c = 1/2$ y $k = 1$.
Cota inferior: para $n \geq 2$ se tiene $n^2 \geq 2n$, es decir, $2n^2 - 2n \geq n^2$, y dividiendo entre 4,

$$\frac{n(n-1)}{2} \geq \frac{n^2}{4},$$

así que $C(n) \geq c\, n^2$ con $c = 1/4$ y $k = 2$.

Conclusión

Por lo tanto, se puede concluir que $C(n) \in O(n^2)$ con testigos $c = 1/2$ y $k = 1$, y $C(n) \in \Omega(n^2)$ con testigos $c = 1/4$ y $k = 2$: $C(n) \in \Theta(n^2)$.

El peor caso: ya ordenado IV

La entrada más fácil para insertion sort

Un arreglo ya ordenado le cuesta a insertion sort $n - 1$ comparaciones y a quicksort $n(n-1)/2$. Con el pivote en el último lugar, ordenar algo que ya está ordenado es el peor caso. Al revés pasa lo mismo: el pivote es el mínimo y cada llamada vuelve a quitar uno solo.

La pila de llamadas I

Quicksort no pide arreglo auxiliar, pero cada llamada pendiente ocupa un registro en la pila de llamadas, con `a`, `ini`, `fin` y `q`. El espacio aparte es la profundidad máxima de la recursión.

Caso	Profundidad	Por qué
balanceado	$\Theta(\log n)$	cada nivel parte el rango por la mitad
ya ordenado	$\Theta(n)$	cada nivel quita un elemento

Lo que se ve al correrlo

Sobre un arreglo ya ordenado queda una llamada pendiente por elemento. Con la pila de llamadas de 8 MB que da Linux por omisión, el programa ordena 100 000 elementos y se cae con 300 000, por desbordar la pila antes de terminar.

Escoger otro pivote I

El peor caso aparece porque $a[fin]$ es el máximo o el mínimo del rango. El pivote no tiene que ser el último: basta escoger otra casilla e intercambiarla con $a[fin]$ antes de partir, y `particionar` queda igual.

- **El del medio**: intercambiar $a[(ini + fin)/2]$ con $a[fin]$. Sobre un arreglo ya ordenado el pivote es la mediana y las particiones quedan balanceadas. Existen otras entradas que lo hacen cuadrático, pero no son las que aparecen solas.
- **Al azar**: escoger la casilla con un número aleatorio entre ini y fin. Ninguna entrada fija provoca siempre el peor caso, y el tiempo esperado es $\Theta(n \log n)$ sobre cualquier entrada (CLRS, §7.3 y §7.4). El peor caso sigue siendo $\Theta(n^2)$, pero depende de la suerte y no de los datos.

Los tres algoritmos sobre la misma entrada I

Comparaciones contadas con un mismo programa: insertion sort y quicksort sobre el arreglo, merge sort sobre la lista enlazada. El mezclado toma valores entre 0 y 10 006 sin orden.

Entrada	n	insertion sort	merge sort	quicksort
mezclado	16	78	48	59
	128	4 166	740	811
	1024	268 112	8 584	11 414
	4096	4 162 689	43 220	54 266
ordenado	16	15	32	120
	128	127	448	8 128
	1024	1 023	5 120	523 776
	4096	4 095	24 576	8 386 560
al revés	16	135	32	120
	128	8 255	448	8 128
	1024	524 799	5 120	523 776
	4096	8 390 655	24 576	8 386 560

Los tres algoritmos sobre la misma entrada II

- Sobre el mezclado, quicksort hace 54 266 comparaciones con 4096 elementos, 1,26 veces las de merge sort, y unas 77 veces menos que insertion sort.
- Sobre el ordenado los papeles se invierten: insertion sort hace 4 095 y quicksort 8 386 560, que es $4096 \cdot 4095/2$, la cifra del teorema.
- Merge sort no cambia de orden de magnitud con la entrada: parte sin mirar los datos.

Los tres algoritmos sobre la misma entrada III

Lo que la tabla no cuenta

Quicksort hace más comparaciones que merge sort sobre el mezclado, y aun así CLRS (cap. 7) lo presenta como la opción práctica más común para ordenar: sobre un arreglo no reserva nodos, no sigue punteros y recorre casillas vecinas. Contar comparaciones compara algoritmos, no programas.

Los tres, lado a lado I

Algoritmo	Mejor	Peor	Espacio aparte	En el sitio	Por qué
insertion, arreglo	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	sí	corre lo anterior; si ya está ordenado, nada
merge, lista	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(\log n)$	sí	parte por la mitad sin mirar los datos
merge, arreglo	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	no	mezcla en un arreglo auxiliar
quicksort, arreglo	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$ a $\Theta(n)$	sí	las partes dependen del pivote

En merge sort sobre la lista y en quicksort, el espacio aparte es la pila de llamadas. Sobre la lista, merge sort no copia datos: empalma nodos.

Los tres, lado a lado II

Cuál escoger

- Pocos datos, o datos casi ordenados: insertion sort.
- Datos en una lista enlazada, o un peor caso garantizado: merge sort.
- Un arreglo sin orden previo y sin memoria de sobra: quicksort con un pivote que no sea siempre el último.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Para el juez: UVa 10041 — Vito's Family I

UVa 10041 — Vito's Family,
`https://onlinejudge.org/external/100/10041.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=982`

Qué pide

Vito tiene r parientes en una misma avenida, con $0 < r < 500$, y cada uno vive en un número s_i con $0 < s_i < 30\,000$. Hay que escoger el número de su casa para que la suma de las distancias $|s_i - s_j|$ a todos sea mínima, e imprimir esa suma. La primera línea trae el número de casos.

Entradas y salidas del ejemplo

Con $\langle 2, 4\rangle$ la respuesta es 2: cualquier casa entre 2 y 4 suma 2. Con $\langle 2, 4, 6\rangle$ es 4: la casa en el 4 suma $2 + 0 + 2$, y en el 3 sumaría $1 + 1 + 3 = 5$.

Para el juez: UVa 10041 — Vito's Family II

Cómo atacarlo con lo de hoy

La casa óptima es la mediana: si hay más parientes a la derecha que a la izquierda, correrse un paso a la derecha baja la suma. Se ordena con quicksort, se toma $s[r/2]$ y se suman las distancias. Cuesta $\Theta(r \log r)$ esperado y $\Theta(r^2)$ en el peor caso, que con $r < 500$ entra igual.

Las trampas

- Los números no vienen ordenados.
- El promedio no sirve: con $\langle 1, 1, 1, 1, 1, 1000 \rangle$ el promedio es 167,5 y la mediana en 1, que suma 999.
- Varios parientes pueden vivir en el mismo número, y la casa puede coincidir con la de uno de ellos.

Para el juez: UVa 10905 — Children's Game I

UVa 10905 — Children's Game,
`https://onlinejudge.org/external/109/10905.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1846`

Qué pide

Cada caso trae $N \leq 50$ enteros positivos. Hay que pegarlos uno detrás de otro, en algún orden, para formar el entero más grande posible, e imprimirlo. La entrada termina con $N = 0$.

Entradas y salidas del ejemplo

Con 123, 124, 56 y 90 la respuesta es 9056124123. Si se agrega un 9, es 99056124123: el 9 va antes que el 90. Con cinco nueves es 99999.

Para el juez: UVa 10905 — Children's Game II

Cómo atacarlo con lo de hoy

`particionar` solo pregunta si un elemento va antes que el pivote. Los números se leen como texto con `scanf("%s")`, y `vaAntes(p, q)` pega p con q y q con p y compara los dos resultados con `strcmp`. La condición $a[j] \leq x$ se cambia por `!vaAntes(x, a[j])`, y se ordena un arreglo de índices para no copiar cadenas al intercambiar. Con $N \leq 50$, aun el peor caso de quicksort son 1225 comparaciones por caso.

Las trampas

- Ordenar por valor no sirve: 90 es mayor que 9, pero 990 gana a 909.
- Ordenar el texto de mayor a menor tampoco: 121 va después de 12 en ese orden, y 12121 gana a 12112, así que el 12 va primero.
- El enunciado no acota los dígitos: el resultado no cabe en un entero y se arma como texto.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Ejercicio en parejas: todos iguales I

Ejecute `quicksort` sobre $\langle 4, 4, 4, 4, 4 \rangle$.

Las preguntas

1. ¿Qué devuelve la primera llamada a `particionar`?
2. ¿De qué tamaño son las dos llamadas que siguen?
3. ¿Cuántas comparaciones hace en total con n elementos iguales? ¿Ayuda tomar el pivote del medio?

Ejercicio en parejas: todos iguales II

Lo que debe salir

Todos cumplen $a[j] \leq 4$, así que `i` avanza con `j` y `particionar` devuelve $q = fin$: la izquierda tiene $n - 1$ elementos y la derecha ninguno. Es el mismo caso del arreglo ordenado, $n(n-1)/2$ comparaciones, $\Theta(n^2)$.

El pivote del medio no ayuda: todos los candidatos valen lo mismo.

El problema está en la condición $a[j] \leq x$, que manda a todos los iguales al mismo lado.

Ejercicio en parejas: solo un lado I

Se quiere el k-ésimo menor de un arreglo, contando desde 0, sin ordenarlo entero. Después de `particionar`, el pivote ya está en la casilla q que ocuparía con el arreglo ordenado.

Las preguntas

1. Si $k = q$, ¿cuál es la respuesta? ¿Y si $k < q$, o si $k > q$, en qué rango hay que seguir buscando?
2. Escriba `seleccionar(a, ini, fin, k)` con un `while`, sin recursión.
3. ¿Cuánto cuesta si cada partición deja la mitad? ¿Y en el peor caso?

Ejercicio en parejas: solo un lado II

Lo que debe salir

Si $k = q$, la respuesta es $a[q]$. Si $k < q$, el k-ésimo está en $a[ini..q-1]$, y si $k > q$ en $a[q+1..fin]$. A diferencia de quicksort, se sigue por un solo lado: el ciclo mueve `fin` a $q-1$ o `ini` a $q+1$ y vuelve a particionar, hasta que $q = k$.

Con particiones a la mitad el trabajo es $n + n/2 + n/4 + \cdots \leq 2n$: $\Theta(n)$, menos que ordenar. En el peor caso cada partición quita un solo elemento y vuelve a ser $\Theta(n^2)$. El espacio aparte es $\Theta(1)$, porque no hay recursión.

Propuestos para la casa I

1. **De mayor a menor.** ¿Qué línea de `particionar` se cambia para que quicksort ordene de mayor a menor? ¿Cómo quedan las zonas?
2. **La traza al revés.** Siga quicksort sobre $\langle 5, 4, 3, 2, 1 \rangle$. ¿Dónde queda cada pivote y cuántas comparaciones hace en total?
3. **Menos pila.** Cambie la segunda llamada recursiva por un `while` que siga con el rango más grande y llame recursivamente solo con el más pequeño. ¿Qué profundidad alcanza la pila en el peor caso?
4. **Quicksort sobre la lista enlazada.** `i` y `j` solo avanzan. ¿Se puede escribir `particionar` sobre una lista simple, con punteros en lugar de índices? ¿Qué pivote es barato de tomar?
5. **Tres zonas.** Escriba un `particionar` que deje los menores, después los iguales al pivote y al final los mayores, y que devuelva dónde empiezan y terminan los iguales.

Pistas para los propuestos I

1. Solo cambia la comparación: $a[j] \geq x$. Las zonas se voltean: a la izquierda quedan los mayores o iguales y a la derecha los menores.
2. El primer pivote es el 1, el mínimo: queda en la casilla 0 y la derecha tiene cuatro elementos, ya no en orden decreciente. Siga la traza: el total coincide con el del arreglo ordenado.
3. Siempre se llama con el rango pequeño, que tiene a lo sumo la mitad de los elementos. Piense cuántas veces se puede partir n por la mitad antes de llegar a 1.
4. Sí: los dos punteros avanzan de a un nodo, como `i` y `j`. El pivote barato es el primer nodo; los intercambios pueden cambiar datos sin tocar los enlaces.
5. Este queda abierto: cuatro zonas en vez de tres, con un índice más para marcar dónde empieza cada una.

Resumen I

- La cola doble necesita la lista doblemente enlazada con cabeza y `ultimo`: quitar el último exige llegar al penúltimo, y eso solo lo da `anterior`.
- Sin contador, `vacia` sigue en $\Theta(1)$ y `tamano` pasa a $\Theta(n)$. Invertir una cola con una pila cuesta $\Theta(n)$ en tiempo y en espacio.
- `particionar` deja el pivote en su casilla definitiva, con los menores o iguales a la izquierda y los mayores a la derecha, en $\Theta(n)$ y sin arreglo auxiliar.
- Quicksort paga al partir y no paga al combinar: $\Theta(n \log n)$ si las particiones son balanceadas, $\Theta(n^2)$ si el pivote es siempre el máximo o el mínimo, como con el pivote en el último lugar sobre un arreglo ya ordenado.

Resumen II

Lo que sigue

Los contenedores `list`, `deque`, `stack` y `queue` de la biblioteca: qué implementación tiene cada uno por dentro y qué cuestan sus operaciones, comparados con los TAD construidos a mano.

Referencias I


T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 7, quicksort: §7.1 descripción y `PARTITION`, §7.2 desempeño, §7.3 versión aleatoria, §7.4 análisis.


R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 14, búsqueda y ordenamiento.


N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.